

HTML

1. Metadata

- **Definition:** Metadata is information about the data (e.g., description, keywords, character encoding).
- **Example:** `<meta charset="UTF-8">` specifies the character encoding for the document.

2. HTML Forms

- **Attributes:**
 - **action:** Specifies the URL to send the form data to.
 - Example: `<form action="submit.php">`.
 - **method:** Defines the HTTP method (GET or POST).
 - Example: `<form method="POST">`.
 - **enctype:** Specifies how form data should be encoded when sent to the server.
 - Example: `<form enctype="multipart/form-data">` (used for file uploads).
- **Required Field:** The required attribute ensures a field must be filled out.
 - Example: `<input type="text" required>`.

3. Semantic Elements

- `<footer>`: Contains footer information.
- `<header>`: Represents introductory content or navigational links.
- `<aside>`: For tangential content like sidebars or related links.
- `<main>`: Highlights the main content of the document.

4. Responsive Images

- **<picture> Tag:**
 - Allows specifying different image sources for different screen sizes or resolutions.
 - Example:
 - `<picture>`
 - `<source srcset="image-large.jpg" media="(min-width: 800px)">`
 - ``
 - `</picture>`
- **srcset Attribute:** Defines multiple image sizes for an `<img>` element.
 - Example: `<img srcset="image1.jpg 1x, image2.jpg 2x">`.

5. Default Behavior

- Missing `<!DOCTYPE html>`: The browser switches to "quirks mode," causing legacy rendering issues.
- Inline Frames: Use `<iframe>` to embed content within a page.

6. Accessibility

- Use `alt` attribute in `<img>` for descriptive text.
 - Example: ``.
- Use semantic elements (`<header>`, `<main>`, `<footer>`, `<nav>`) for better screen reader support.
- Ensure form elements have labels for accessibility.
 - Example: `<label for="username">Username:</label> <input id="username">`.

CSS

1. Box Model

- The CSS Box Model defines the structure of an HTML element:
 - **Content:** The actual content of the box.
 - **Padding:** Space between the content and the border.
 - **Border:** Surrounds the padding.
 - **Margin:** Space outside the border.
- Order: content > padding > border > margin.

2. Selectors

- `#id`: Selects an element with a specific ID. Example: `#header { color: blue; }`.

- **.class:** Selects all elements with a specific class. Example: `.box { margin: 10px; }`.
- **element:** Selects all elements of a type. Example: `p { line-height: 1.5; }`.

3. Flexbox

- **Properties:**
 - `justify-content:` Aligns items horizontally.
 - `align-items:` Aligns items vertically.
 - `flex-wrap:` Allows items to wrap onto the next line.
- **Example:**
- `.container {`
- `display: flex;`
- `justify-content: space-between;`
- `align-items: center;`
- `}`

4. Grid

- **Properties:**
 - `grid-template-columns:` Defines column sizes.
 - `grid-gap:` Specifies space between rows and columns.
 - `minmax():` Sets a range for column/row sizes.
- **Example:**
- `.grid {`
- `display: grid;`
- `grid-template-columns: repeat(3, 1fr);`
- `gap: 20px;`
- `}`

5. Transitions and Animations

- **Transition:** Smoothly changes property values.
 - Example: `transition: all 0.3s ease-in-out;`
- **Keyframes:** Define animations.
 - Example:
 - `@keyframes slideIn {`
 - `from { transform: translateX(-100%); }`
 - `to { transform: translateX(0); }`
 - `}`

6. Media Queries

- **Definition:** Allows CSS rules to apply based on device properties (e.g., screen size).
- **Example:**
- `@media (max-width: 768px) {`
- `.container {`
- `flex-direction: column;`
- `}`
- `}`

JavaScript

1. ES6 Features

- **let and const:** Block-scoped variables.
- **Arrow Functions:** Concise function syntax. Example: `const sum = (a, b) => a + b;`
- **Promises:** Manage asynchronous operations. Example:
- `fetch('api/data')`
- `.then(response => response.json())`
- `.then(data => console.log(data))`
- `.catch(error => console.error(error));`

2. DOM Manipulation

- `querySelector`: Selects the first matching element.
- `addEventListener`: Adds an event listener to an element.
 - Example:
 - `document.querySelector('button').addEventListener('click', () => {`
 - `alert('Button clicked!');`
 - `});`
- `innerHTML`: Changes an element's content.

3. Events

- `preventDefault()`: Prevents default behavior (e.g., form submission).
- `stopPropagation()`: Stops event bubbling.

4. Error Handling

- **Try-Catch:**
- `try {`
- `// Code that might throw an error`
- `} catch (error) {`
- `console.error(error);`
- `}`

5. Useful Array Methods

- `map()`: Creates a new array by applying a function to each element.
- `filter()`: Creates a new array with elements that pass a condition.
- `reduce()`: Reduces an array to a single value by applying a function.

Bootstrap

1. Grid System

- Divide the screen into 12 columns.
- **Classes:**
 - `.col-:` Defines column sizes.
 - `.row:` Creates a row.
 - `.container:` Creates a centered, fixed-width container.

2. Components

- Buttons: `.btn` classes, e.g., `.btn-primary`.
- Cards: `.card` for card components.
 - Example:
 - `<div class="card">`
 - `<div class="card-body">This is a card.</div>`
 - `</div>`

3. Utilities

- Centering Text: `.text-center`.
- Shadows: `.shadow-sm`, `.shadow-lg`.
- Responsive Tables: `.table-responsive`.
- Visibility: `.d-none`, `.d-block`.

React

1. Hooks

- **useState**: Manages state in functional components.
 - Example:
 - `const [count, setCount] = useState(0);`
- **useEffect**: Handles side effects like API calls.
 - Example:
 - `useEffect(() => {`

- `console.log('Component mounted');`
- `}, []);`

2. Optimization

- `React.memo`: Prevents unnecessary re-renders.
- `useMemo`: Caches computed values.

3. Props and State

- `props`: Data passed from parent to child components.
 - Example:
 - `function Greeting({ name }) {`
 - `return <h1>Hello, {name}</h1>;`
 - `}`
- `key Prop`: Helps React identify updated elements in lists.

4. Conditional Rendering

- Use ternary operators or `&&` in JSX.
 - Example:
 - `return isLoggedIn ? <Dashboard /> : <Login />;`

jQuery

1. Common Methods

- `.hide()`: Hides elements.
- `.fadeOut()`: Fades elements out.
- `.append()`: Adds content at the end of an element.

2. Event Handling

- `.on()`: Binds an event to an element.
 - Example:
 - `$('#button').on('click', () => {`
 - `alert('Button clicked!');`
 - `});`
- `.off()`: Removes an event handler.

3. DOM Traversal

- `.children()`: Selects direct children.
- `.next()`, `.prev()`: Navigate siblings.

4. Animations

- `.animate()`: Custom animations.
 - Example:
 - `$('#box').animate({ height: '300px' }, 500);`

This comprehensive guide covers essential topics and their applications to help you excel in your Web Engineering final exam. Focus on practicing examples to reinforce your understanding.

HTML (HyperText Markup Language)

1. Basic Structure of an HTML Document

- **<!DOCTYPE html>**: Declares the document type and version of HTML (HTML5 in this case).
- **<html>**: The root element of an HTML document.
- **<head>**: Contains meta-information about the document, such as the title, character set, and links to external resources.
 - **<meta charset="UTF-8">**: Specifies the character encoding for the document.
 - **<title>**: Sets the title of the webpage (displayed in the browser tab).
- **<body>**: Contains the visible content of the webpage.

2. Semantic HTML5 Tags

- **<header>**: Represents the header of a section or webpage.
- **<footer>**: Represents the footer of a section or webpage.
- **<article>**: Represents a self-contained piece of content (e.g., a blog post).
- **<section>**: Represents a thematic grouping of content.
- **<nav>**: Represents a section of navigation links.
- **<aside>**: Represents content that is tangentially related to the main content (e.g., a sidebar).

3. Forms and Input Elements

- **<form>**: Used to create a form for user input.
 - **action**: Specifies where to send the form data (URL).
 - **method**: Specifies how to send the form data (GET or POST).
 - **enctype**: Specifies how the form data should be encoded (e.g., multipart/form-data for file uploads).
- **<input>**: Used to create various input fields.
 - **type="text"**: Text input.
 - **type="password"**: Password input (characters are masked).
 - **type="submit"**: Submit button.
 - **type="file"**: File upload input.
- **<textarea>**: Multi-line text input.
- **<select> and <option>**: Dropdown list.
- **<button>**: Clickable button.

4. HTML5 Features

- **<meter>**: Represents a scalar measurement within a known range (e.g., disk usage).
- **<progress>**: Represents the progress of a task (e.g., file upload).
- **<picture>**: Allows you to specify multiple image sources for different screen sizes.
- **<video> and <audio>**: Embed video and audio content.
- **<canvas>**: Used for drawing graphics via JavaScript.

5. Attributes

- **required**: Specifies that an input field must be filled out before submitting the form.
- **pattern**: Specifies a regular expression that the input field's value must match.
- **placeholder**: Provides a hint to the user about what to enter in an input field.
- **autofocus**: Automatically focuses on the input field when the page loads.

CSS (Cascading Style Sheets)

1. Selectors

- **Class Selector**: `.classname { ... }` (e.g., `.container { ... }`)
- **ID Selector**: `#idname { ... }` (e.g., `#header { ... }`)
- **Tag Selector**: `tagname { ... }` (e.g., `p { ... }`)
- **Descendant Selector**: `div p { ... }` (selects all `<p>` elements inside a `<div>`)
- **Child Selector**: `div > p { ... }` (selects direct child `<p>` elements of a `<div>`)

2. Box Model

- **Content:** The actual content of the element (text, images, etc.).
- **Padding:** Space between the content and the border.
- **Border:** A line surrounding the padding and content.
- **Margin:** Space outside the border, separating the element from other elements.
- **box-sizing: border-box;** Includes padding and border in the element's total width and height.

3. Positioning

- **position: static;** Default positioning (elements flow in the normal document flow).
- **position: relative;** Positions the element relative to its normal position.
- **position: absolute;** Positions the element relative to the nearest positioned ancestor.
- **position: fixed;** Positions the element relative to the viewport (stays fixed even when scrolling).
- **position: sticky;** Combines relative and fixed positioning (sticks to the viewport when scrolling).

4. Flexbox

- **display: flex;** Turns an element into a flex container.
- **flex-direction:** Defines the direction of the flex items (row, column, row-reverse, column-reverse).
- **justify-content:** Aligns items along the main axis (e.g., center, space-between).
- **align-items:** Aligns items along the cross axis (e.g., center, stretch).
- **flex-wrap:** Allows flex items to wrap onto multiple lines if necessary.

5. Grid Layout

- **display: grid;** Turns an element into a grid container.
- **grid-template-columns:** Defines the columns in the grid (e.g., 1fr 1fr 1fr for three equal columns).
- **grid-template-rows:** Defines the rows in the grid.
- **grid-gap:** Defines the gap between grid items.

6. Styling Properties

- **background-color:** Sets the background color of an element.
- **color:** Sets the text color.
- **font-size:** Sets the size of the text.
- **text-align:** Aligns text (e.g., center, left, right).
- **z-index:** Controls the stacking order of elements (higher values appear on top).

JavaScript & ES6

1. Variables and Data Types

- **let:** Block-scoped variable (can be reassigned).
- **const:** Block-scoped constant (cannot be reassigned).
- **Primitive Data Types:** string, number, boolean, null, undefined, symbol.
- **Objects:** Collections of key-value pairs.

2. Functions

- **Arrow Functions:** Shorter syntax for writing functions.

```
javascript
```

Copy

```
const add = (a, b) => a + b;
```

- **Default Parameters:** Allows you to set default values for function parameters.

```
javascript
```

Copy

```
function greet(name = "Guest") {  
  return `Hello, ${name}`;  
}
```

3. Promises and Async/Await

- **Promises:** Used for handling asynchronous operations.

javascript

Copy

```
const promise = new Promise((resolve, reject) => {  
  if (success) {  
    resolve("Success!");  
  } else {  
    reject("Error!");  
  }  
});
```

- **Async/Await:** Simplifies working with promises.

javascript

Copy

```
async function fetchData() {  
  const response = await fetch('url');  
  const data = await response.json();  
  return data;  
}
```

4. DOM Manipulation

- **Selecting Elements:**

javascript

Copy

```
document.getElementById('id');  
document.querySelector('.class');  
document.querySelectorAll('p');
```

- **Modifying Elements:**

javascript

Copy

```
element.innerHTML = 'New content';  
element.style.color = 'red';  
element.classList.add('new-class');
```

- **Event Handling:**

javascript

Copy

```
element.addEventListener('click', function() {  
  alert('Clicked!');  
});
```

Bootstrap

1. Grid System

- **.container**: Fixed-width container.
- **.container-fluid**: Full-width container.
- **.row**: Creates a row for columns.
- **.col-md-6**: Creates a column that takes up 6 out of 12 grid spaces on medium screens.

2. Components

- **Navbar**: .navbar, .navbar-expand-md
- **Cards**: .card, .card-body, .card-header
- **Buttons**: .btn, .btn-primary, .btn-secondary
- **Modals**: .modal, .modal-dialog, .modal-content

3. Utilities

- **Spacing**: .p-3 (padding), .m-0 (margin)
- **Text Alignment**: .text-center, .text-right
- **Visibility**: .d-none (hide), .d-block (show)

React

1. Components

- **Functional Components:**

```
javascript
```

Copy

```
function Welcome(props) {  
  return <h1>Hello, {props.name}</h1>;  
}
```

- **Class Components:**

```
javascript
```

Copy

```
class Welcome extends React.Component {  
  render() {  
    return <h1>Hello, {this.props.name}</h1>;  
  }  
}
```

2. State and Props

- **useState Hook:**

```
javascript
```

Copy

```
const [count, setCount] = useState(0);
```

- **Props**: Pass data from parent to child components.

```
javascript
```

Copy

```
function Child(props) {  
  return <p>{props.message}</p>;  
}
```

```
}
```

3. Lifecycle Methods and Hooks

- **useEffect Hook:** Handles side effects (e.g., fetching data).

```
javascript
```

Copy

```
useEffect(() => {  
  fetchData();  
}, []);
```

4. Routing

- **React Router:**

```
javascript
```

Copy

```
<Router>  
  <Route path="/about" component={About} />  
  <Route path="/contact" component={Contact} />  
</Router>
```